

[40] Song: Approximate nearest neighbor search on gpu

저자: W. Zhao, S. Tan, and P. Li

학술지: ICDE 2020, pp. 1033-1044

주제: GPU 기반의 근사 근접 이웃 검색

요약

본 논문은 GPU(Graphics Processing Unit)를 활용한 고속 ANN(근사 근접 이웃) 검색 알고리즘 SONG(Spatial Ordered Navigation Graph)을 제시한다. GPU의 대규모 병렬 처리 능력을 활용하여 벡터 공간에서의 효율적인 검색을 구현한다.

SONG의 핵심은 공간 정렬(spatial ordering)을 통해 벡터들을 구조화하고, GPU 메모리 계층을 고려한 최적화된 네비게이팅 그래프를 구성하는 것이다. 배치 쿼리 처리와 대규모 병렬화로 CPU 기반 방식보다 훨씬 빠른 성능을 달성한다.

본 논문과의 관계: Exqutor가 필터링 기반 벡터 검색을 GPU에서 구현할 경우, SONG의 공간 정렬과 GPU 최적화 기법을 활용할 수 있다. 특히 대규모 쿼리 배치 처리 시 필터링과 거리 계산을 GPU에서 병렬로 수행하는 전략이 효과적이다.

상세분석

40.1 GPU 기반 ANN의 필요성

CPU 기반의 한계:

- 단일 스레드 성능 한계에 도달
- 벡터 거리 계산의 높은 CPU 비용
- 메모리 대역폭 부족으로 성능 정체

GPU의 장점:

- 수천 개의 코어를 통한 대규모 병렬 처리
- 고대역폭 메모리 시스템 (HBM, GDDR)
- 벡터 연산(SIMD)에 특화된 하드웨어
- 배치 처리에 최적화

활용 시나리오:

- 대규모 배치 검색 (수천~수십만 쿼리 동시 처리)
- 실시간 인터랙티브 응용
- 추천 시스템의 후보 선택

40.2 GPU 메모리 계층과 최적화

GPU 메모리 구조:

레지스터 (Register)
↓ (매우 빠름, 제한적)
공유 메모리 (Shared Memory)
↓ (빠름, 블록 내 공유)
글로벌 메모리 (Global Memory)
↓ (느림, 높은 지연)
메인 메모리 (Host Memory)

최적화 전략:

1. 데이터 지역성: 자주 접근되는 데이터를 공유 메모리에 캐싱
2. 메모리 접근 패턴: 연속적 메모리 접근으로 높은 대역폭 활용
3. 계산과 전송 오버래핑: 데이터 전송 중 계산 실행

40.3 SONG 알고리즘 - 공간 정렬

공간 정렬(Spatial Ordering)의 개념:

- 벡터들을 공간상의 순서에 따라 정렬
- 유사한 벡터들이 메모리상에 인접하게 배치
- 캐시 효율성과 메모리 접근성 향상

공간 정렬 방법:

1. Z-order curve (Morton order) 사용
 - 공간을 재귀적으로 분할하며 점들을 순서화
 - 다차원 공간을 1차원으로 인코딩
2. Hilbert curve 사용
 - Z-order보다 공간 지역성 더 잘 보존
 - 더 나은 클러스터링 구조
3. 쿼드트리 기반 정렬
 - 계층적 공간 분할
 - GPU 병렬화에 적합

Z-order 인덱싱 예시 (2D):

공간상의 점:

- (1, 1) → Z-인덱스 6
- (0, 1) → Z-인덱스 4
- (0, 0) → Z-인덱스 0
- (1, 0) → Z-인덱스 2

Z-순서: (0,0) → (1,0) → (0,1) → (1,1)

40.4 네비게이팅 그래프 구성

정렬된 벡터로부터 그래프 생성:

```
// 단계 1: 벡터들을 공간 순서대로 정렬
sorted_vectors = spatial_sort(all_vectors)

// 단계 2: 각 벡터에 대해 이웃 연결
for each vector v in sorted_vectors:
    // 로컬 이웃 (공간 순서상 근접)
    local_neighbors = get_spatial_neighbors(v)

    // 글로벌 이웃 (거리 기준)
    global_neighbors = search_nearest(v, K)

    // 하이브리드 이웃 집합
    neighbors[v] = combine(local_neighbors,
                           global_neighbors)

// 단계 3: GPU 최적화를 위한 그래프 리정렬
reorder_for_gpu_cache(graph)
```

40.5 GPU 기반 배치 검색

병렬 검색 커널:

```

__global__ void search_batch_kernel(
    const float* queries,      // N_q 개 쿼리
    const float* vectors,     // N 개 벡터
    const int* graph,         // 이웃 관계
    int K, int ef,
    int* results               // 결과 저장
) {
    int query_id = blockIdx.x; // 쿼리별 블록
    int thread_id = threadIdx.x; // 스레드

    // 공유 메모리에 쿼리 벡터 로드
    __shared__ float query[VECTOR_DIM];
    if (thread_id < VECTOR_DIM) {
        query[thread_id] = queries[query_id * VECTOR_DIM
                                   + thread_id];
    }
    __syncthreads();

    // 각 스레드가 별도의 벡터와 거리 계산
    int vector_id = thread_id;
    while (vector_id < N) {
        float dist = compute_distance(
            query,
            vectors[vector_id],
            VECTOR_DIM
        );

        // 결과 업데이트 (원자적 연산)
        atomicUpdate(results, query_id,
                     vector_id, dist);

        vector_id += blockDim.x; // 그리드 스트라이드
    }
}

```

40.6 거리 계산의 병렬화

벡터 거리 계산:

```
// 유클리드 거리
distance = sqrt(sum((q_i - v_i)^2))

// GPU에서 병렬 계산
__device__ float compute_l2_distance(
    const float* q, const float* v, int dim
) {
    float sum = 0.0f;

    for (int i = threadIdx.x; i < dim;
        i += blockDim.x) {
        float diff = q[i] - v[i];
        sum += diff * diff;
    }

    // 블록 내 리덕션
    return block_reduce_sum(sum); // sum 병렬 합산
}
```

코사인 유사도:

```
similarity = dot(q, v) / (||q|| * ||v||)

// GPU 최적화
// 1. 벡터 정규화 사전 계산
// 2. 내적 계산 병렬화
// 3. 시뮬레이션된 거리 계산 (1 - similarity)
```

40.7 배치 처리 최적화

배치 크기 선택:

배치 크기 = GPU 메모리 / (쿼리당 메모리 + 임시 메모리)

예: GPU 메모리 32GB

쿼리 당 메모리 = 벡터크기 + 결과 + 임시 버퍼
 $\approx 8\text{KB}$ (벡터 차원 1000)

최적 배치 크기 $\approx 4,000,000$ 쿼리

배치 파이프라이닝:

```
시간 →
[배치 1]
  H2D 전송 (호스트→GPU)
  └─ GPU 계산 (겹침)
  └─ D2H 전송 (GPU→호스트) [겹침]
[배치 2]
  H2D 전송 [겹침]
  └─ GPU 계산
  └─ D2H 전송 [겹침]
```

40.8 메모리 효율성

메모리 사용량 분석:

벡터 데이터: $N * \text{dim} * 4$ 바이트
 그래프 구조: $N * M * 4$ 바이트 (M = 이웃 수)
 정렬 인덱스: $N * 8$ 바이트

총합: $N * (4 * \text{dim} + 4 * M + 8)$ 바이트

예: 100만 벡터, 1000차원, $M=16$
 $= 1M * (4K + 64 + 8) = \text{약 } 4GB$

GPU 메모리 제약:

- 고급 GPU (A100): 40~80GB
- 중급 GPU (RTX): 8~24GB
- 일반 GPU (GTX): 2~6GB

40.9 CPU와 GPU 성능 비교

지표	CPU (HNSW)	GPU (SONG)
단일 쿼리	매우 빠름	중간 (오버헤드)
배치 100	빠름	매우 빠름
배치 10K	중간	극도로 빠름
메모리	적음	많음
전력 효율	우수	낮음
비용	낮음	높음

성능 향상:

- 배치 쿼리: 10~100배 빠름
- 높은 처리량: 초당 백만 쿼리 처리 가능

40.10 필터링과의 통합

GPU 기반 필터링:

```

__global__ void search_filtered_kernel(
    const float* queries,
    const float* vectors,
    const int* attributes,      // 속성값
    const AttributeFilter* filters,
    int* results
) {
    int query_id = blockIdx.x;
    int vector_id = threadIdx.x;

    // 1. 필터 조건 확인 (GPU에서 빠름)
    if (!check_filter(attributes[vector_id], filters)) {
        return; // 필터링된 벡터 스킵
    }

    // 2. 거리 계산
    float dist = compute_distance(
        queries[query_id],
        vectors[vector_id]
    );

    // 3. 결과 저장
    atomicUpdate(results, query_id, vector_id, dist);
}

```

40.11 SONG의 장점

극도의 처리량:

- 초당 수백만 쿼리 처리
- 대규모 배치 작업에 최적

병렬성:

- 쿼리 수준 병렬화 (각 쿼리가 별도 블록)
- 벡터 수준 병렬화 (각 벡터가 별도 스레드)
- 차원 수준 병렬화 (거리 계산 병렬)

메모리 대역폭 활용:

- GPU 메모리 대역폭: 1~5TB/s (vs CPU 100GB/s)
- 공간 정렬로 캐시 효율성 향상

40.12 SONG의 단점

지연시간(Latency):

- GPU 전송 오버헤드
- 단일 쿼리: CPU보다 느릴 수 있음

메모리 제약:

- 대규모 데이터셋은 여러 GPU 필요
- 정렬에 따른 메모리 재구성 비용

동적 데이터:

- 삽입/삭제 시 공간 정렬 재구성 필요
- 배치 업데이트에만 효율적

추가 제기 문제

1. **필터 조건의 GPU 최적화**: 복잡한 다중 필터 조건을 GPU에서 효율적으로 평가하려면? 필터 조건의 선택도에 따른 성능 예측 모델은?
2. **공간 정렬과 필터 상관성**: 벡터들이 공간상으로 정렬되었을 때, 같은 필터 조건을 만족하는 벡터들도 함께 클러스터링될까?
3. **GPU 메모리 계층의 필터링 활용**: 공유 메모리에 필터 인덱스를 캐싱하면 필터 처리 성능을 얼마나 향상시킬 수 있을까?
4. **다중 GPU 확장**: 여러 GPU에서 필터링된 검색을 분산 처리할 때 최적의 데이터 분할 전략은?
5. **실시간 업데이트와 필터링**: GPU에서 벡터를 실시간으로 추가하면서 필터링 성능을 유지하려면?
6. **필터 선택도와 성능**: 필터 선택도가 높을 때(많은 결과 제외) GPU의 이점이 얼마나 감소할까?
7. **전력 효율**: 필터링을 추가했을 때 GPU의 전력 소비와 처리량의 트레이드오프는?
8. **하이브리드 아키텍처**: CPU와 GPU를 함께 사용하여 필터링된 검색의 지연시간과 처리량을 모두 최적화할 수 있을까?